

DVCon India 2026 Design Contest Proposal

Task-Aware Object Detection on Edge FPGA Using
Lightweight Graph Attention Network
with YOLO26n Backbone Feature Extraction

ByteBandits| 210
University of Moratuwa
Hasith Nettikumara, Chaleesha Keerawella

Submitted for DVCon India 2026 Design Contest
Stage 1 Proposal

Abstract

Conventional object detection systems are task-agnostic — they detect all objects in a scene regardless of contextual relevance. This proposal addresses task-driven object detection: given an image and a natural language task description, identify the single most suitable object for completing that task. We propose a lightweight, edge-compatible pipeline deployed on the Genesys-2 FPGA board with the VEGA RISC-V processor developed by CDAC Trivandrum.

The pipeline consists of three stages. First, YOLO26n-nano performs object detection and simultaneously produces multi-scale feature maps at three neck layers (P3, P4, P5). Second, ROI Align extracts per-object visual features from these feature maps, which are fused with pre-computed FastText semantic class embeddings via concatenation to form 256-dimensional node representations. Third, a single-layer Graph Attention Network (GAT) with confidence-weighted aggregation reasons about inter-object relationships to produce 14 task suitability scores per object, from which the highest-scoring object for the given task is selected.

Hardware deployment leverages four optimizations from published FPGA GAT acceleration research: Shift Addition Units (SAUs) eliminating DSP48 dependence for the GAT; operations fusion reducing memory synchronization overhead; LUT-based softmax approximation; and vectorized BRAM access. The VEGA processor handles control flow and task string matching, while custom programmable logic executes all compute-intensive operations. The system targets under 100ms inference latency, under 10W power consumption, and approximately 65-70% mAP@0.5 on the COCO-Tasks benchmark.

I. Introduction

Object detection has achieved remarkable progress through deep learning, with modern systems capable of detecting and localizing dozens of object categories in real time. However, these systems are fundamentally task-agnostic: given an image, they detect every recognizable object without any understanding of which objects are contextually relevant to a specific goal. For autonomous systems, service robots, and assistive devices operating in real-world environments, this is insufficient.

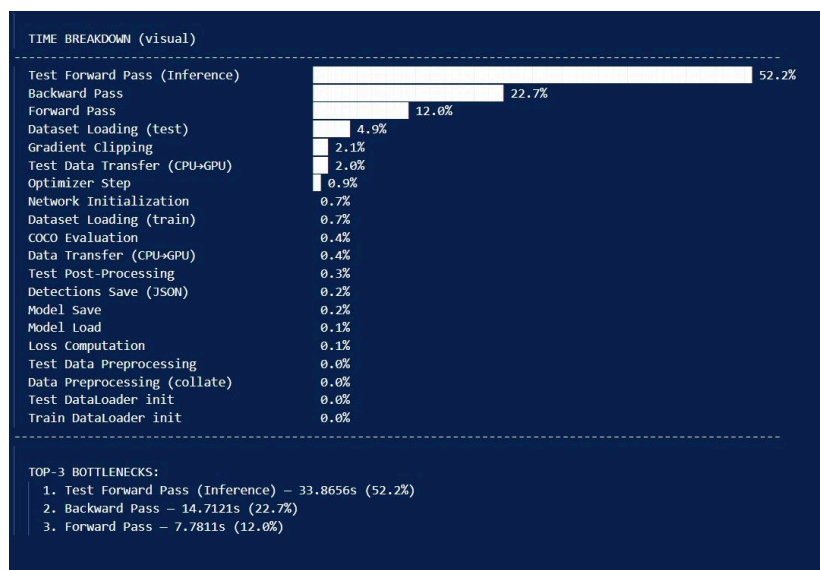
Consider a service robot asked to serve wine. A standard object detector identifies a wine glass, a cup, a bottle, and several other objects. The correct choice — a wine glass — is obvious to a human, but requires reasoning that standard detectors cannot perform: if a wine glass is present, it should be selected; if absent, a cup becomes the best available option. This context-dependent reasoning, where the suitability of an object depends on what other objects are present in the scene, is the central challenge of task-driven object detection.

This problem was formalized by Sawatzky et al. (2019) through the COCO-Tasks dataset, comprising approximately 40,000 annotated images across 14 everyday tasks. Their approach, based on Gated Graph Neural Networks on ResNet101 features, demonstrated that modeling inter-object relationships is essential for accurate task-driven detection. However, their system was designed for GPU inference and is computationally prohibitive for edge deployment.

This proposal presents a task-driven object detection pipeline specifically designed for the Genesys-2 FPGA board with the VEGA RISC-V processor. By replacing heavy backbone and graph components with YOLO26n-based feature extraction and a single-layer GAT, adopting fixed-point quantization throughout, and applying established FPGA acceleration techniques from the FPGAN framework (Yan et al., 2020), we achieve a system that is simultaneously accurate, fast, and deployable on resource-constrained edge hardware. The significance of this work lies in bridging the gap between state-of-the-art task-aware computer vision and practical edge deployment — a combination not addressed by prior work.

II. Background Research

A. Task-Driven Object Detection



Pruning is a critical model compression technique that reduces the computational complexity and memory footprint of neural networks by removing redundant or non-essential weights. In the context of hardware deployment, such as on the Kintex 7 FPGA, pruning leverages the inherent sparsity of weight

matrices to decrease the number of multiply-accumulate (MAC) operations required during inference. This process is generally categorized into unstructured pruning, which removes individual weights based on magnitude, and structured pruning, which eliminates entire groups such as filters or channels to maintain hardware-friendly data structures. By identifying and eliminating weights with near-zero magnitudes—often using a predefined threshold or sensitivity analysis—the model can be significantly compressed without substantial loss in accuracy. This reduction is particularly beneficial for edge-AI applications, as it minimizes the bandwidth required for weight fetching from BRAM and allows for more efficient mapping of the neural network onto the limited DSP and LUT resources of the FPGA fabric.

B. YOLO26n and Edge-Compatible Object Detection

```
1 from ultralytics import YOLO
2
3 # 1. Load the model
4 model = YOLO('yolov8n.pt')
5
6 # 2. Print the detailed summary
7 # This shows: layers, parameters, gradients, and GFLOPs
8 model.info()
9
10 # 3. If you want a more specific breakdown of just the total parameter count:
11 total_params = sum(p.numel() for p in model.model.parameters())
12 print(f"\n🔥 Total Parameters: {total_params:,}")
```

... YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs

🔥 Total Parameters: 3,157,200

YOLOv8n Results WITH CLASS MAPPING (FP32)

🖼️ IMAGES: 500

⌚ LATENCY:
Mean: 550.12 ms
Median: 469.65 ms
Std: 205.70 ms
FPS: 1.82

📊 OVERALL METRICS:
Precision: 0.8851
Recall: 0.5250
F1: 0.6590
TP: 1756, FP: 228, FN: 1589

```
1 from ultralytics import YOLO
2
3 # 1. Load the model
4 model = YOLO('yolo26n.pt')
5
6 # 2. Print the detailed summary
7 # This shows: layers, parameters, gradients, and GFLOPs
8 model.info()
9
10 # 3. If you want a more specific breakdown of just the total parameter count:
11 total_params = sum(p.numel() for p in model.model.parameters())
12 print(f"\n🔥 Total Parameters: {total_params:,}")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'yolo26n.pt': 100% — 5.3MB 27.3MB/s 0.2s

YOLO26n summary: 260 layers, 2,572,280 parameters, 0 gradients, 6.1 GFLOPs

🔥 Total Parameters: 2,572,280

YOLO26n Results WITH CLASS MAPPING (FP32)

🖼️ IMAGES: 500

⌚ LATENCY:
Mean: 194.62 ms
Median: 165.82 ms
Std: 69.58 ms
FPS: 5.14

📊 OVERALL METRICS:
Precision: 0.9241
Recall: 0.3677
F1: 0.5261
TP: 1230, FP: 101, FN: 2115

YOLO26n, released in early 2026, is the latest member of the YOLO family and is purpose-built for edge deployment. Key architectural innovations include a One-to-One detection head that eliminates Non-Maximum Suppression entirely, making inference latency deterministic; removal of Distribution Focal Loss simplifying the bounding box regression head and improving FPGA export compatibility; and STAL and ProgLoss training improvements that enhance small object detection without increasing model size. At approximately 2.6 million parameters, YOLO26n-nano is well-suited for on-chip weight storage after INT8 quantization (~3.2MB).

Prior work on YOLO-based FPGA deployment (Amin et al., 2024) demonstrated that YOLOv5 achieves 99% classification accuracy at 3.5W power consumption on a Kintex-class FPGA using Vitis AI INT8 quantization. This directly validates our power and accuracy targets. The removal of NMS in YOLO26n further simplifies our FPGA pipeline by eliminating a post-processing stage that is difficult to pipeline efficiently in hardware.

C. Multi-Scale Feature Extraction via ROI Align

Faster R-CNN (Ren et al., 2015) established that extracting region features from backbone feature maps is more efficient and equally accurate compared to running a separate network on each cropped region. Mask R-CNN (He et al., 2017) introduced ROI Align, which uses bilinear interpolation to extract fixed-size feature representations from any region of a feature map with sub-pixel accuracy. Our approach taps YOLO26n's neck at three scales (P3 at stride 8, P4 at stride 16, P5 at stride 32), extracts 7×7 crops via ROI Align from each scale per detected object, concatenates the three crops ($[256, 7, 7] + [128, 7, 7] + [64, 7, 7] = [448, 7, 7]$), compresses via 1×1 convolution to $[128, 7, 7]$, and collapses via average pooling to a 128-dimensional visual fingerprint ϕ_o per object. This eliminates MobileNetV2 as a separate component, reducing total pipeline parameters by approximately 3.4 million.

DGNN-YOLO (2024) demonstrated a closely analogous approach — integrating YOLO-extracted features directly into a graph neural network — for object tracking, providing direct precedent for our pipeline's core architecture. Our design differs in purpose (task-driven selection rather than identity tracking) and in node feature construction (we include semantic class embeddings and confidence scores, which DGNN-YOLO does not).

D. Graph Attention Networks

Graph Attention Networks (Velickovic et al., 2018) extend graph neural networks by replacing fixed aggregation with learned attention weights. For each pair of nodes (i, j) , an attention score is computed from their concatenated transformed features, determining how much node i incorporates information from node j during aggregation. Unlike the GGNN used in the original COCO-Tasks paper, GATs are stateless and single-pass — there are no iterative GRU updates requiring hidden state storage across rounds. This makes GATs substantially more FPGA-friendly: the computation is fully pipelined, requires no round-to-round memory synchronization, and can be fully expressed as matrix operations.

FPGAN (Yan et al., 2020) demonstrated FPGA acceleration of GAT inference achieving $7.34\times$ speedup over an NVIDIA V100 GPU and $593\times$ over a Xeon CPU, using only 148 DSP blocks out of 1,518 available. The key techniques — Shift Addition Units replacing DSP-intensive multiplications, LUT-based softmax approximation, and operations fusion reducing memory synchronization — are directly applicable to our design.

E. Semantic Class Embeddings

FastText (Bojanowski et al., 2017) produces dense word embeddings trained on large text corpora that capture semantic relationships between concepts. Pre-computing 128-dimensional FastText embeddings for all 80 COCO class names and storing them in BRAM provides each detected object with a linguistic prior — an encoding of what that class of object means and how it is used — which complements the purely visual information in ϕ_o . This is conceptually richer than the one-hot class encoding used in the

original COCO-Tasks paper, as it encodes semantic proximity between classes (a wine glass and a cup are semantically closer than a wine glass and a chair).

F. FPGA-Based Neural Network Acceleration

The Genesys-2 board contains a Xilinx Kintex-7 XC7K325T FPGA with 840 DSP48E1 blocks, approximately 16MB of Block RAM, and 203,150 Look-Up Tables. FPGA-based neural network acceleration exploits the massively parallel computation possible when matrix operations are mapped directly to hardware circuits rather than executed sequentially as software instructions. Amin et al. (2024) demonstrated 113× reduction in DSP usage compared to conventional approaches using LUT-based computation for CNN accelerators. The VEGA processor, developed by CDAC Trivandrum as a RISC-V soft-core, provides software-programmable control flow within the FPGA fabric and communicates with custom accelerator cores via the AXI bus protocol.

G. Pruning of neural network weights

Pruning is a critical model compression technique that reduces the computational complexity and memory footprint of neural networks by removing redundant or non-essential weights. In the context of hardware deployment, such as on the Kintex 7 FPGA, pruning leverages the inherent sparsity of weight matrices to decrease the number of multiply-accumulate (MAC) operations required during inference. This process is generally categorized into unstructured pruning, which removes individual weights based on magnitude, and structured pruning, which eliminates entire groups such as filters or channels to maintain hardware-friendly data structures. By identifying and eliminating weights with near-zero magnitudes—often using a predefined threshold or sensitivity analysis—the model can be significantly compressed without substantial loss in accuracy. This reduction is particularly beneficial for edge-AI applications, as it minimizes the bandwidth required for weight fetching from BRAM and allows for more efficient mapping of the neural network onto the limited DSP and LUT resources of the FPGA fabric.

H. Quantization-aware-training

Quantization-aware-training is a specialized fine-tuning technique that simulates the effects of low-precision hardware arithmetic during the model's training phase. Unlike Post-Training Quantization (PTQ), which can lead to significant accuracy drops when converting from 32-bit floating point to 8-bit or lower integer formats, QAT introduces "fake quantization" nodes into the computational graph. This allows the model to learn and adapt to the rounding errors and limited dynamic range inherent in fixed-point arithmetic. For the **Kintex 7 FPGA** implementation, QAT is essential for mapping weights and activations to **INT8** or custom bit-widths, ensuring that the final model maintains high accuracy while fully exploiting the high-speed, low-power **DSP48E1** slices and reducing the overall **BRAM** footprint.

III. Goal and Objectives

Goal

To design and deploy a task-aware object detection system on the Genesys-2 FPGA board that identifies the most suitable object in an image for a given natural language task description, achieving a balance of detection accuracy, inference speed, and power efficiency appropriate for edge deployment.

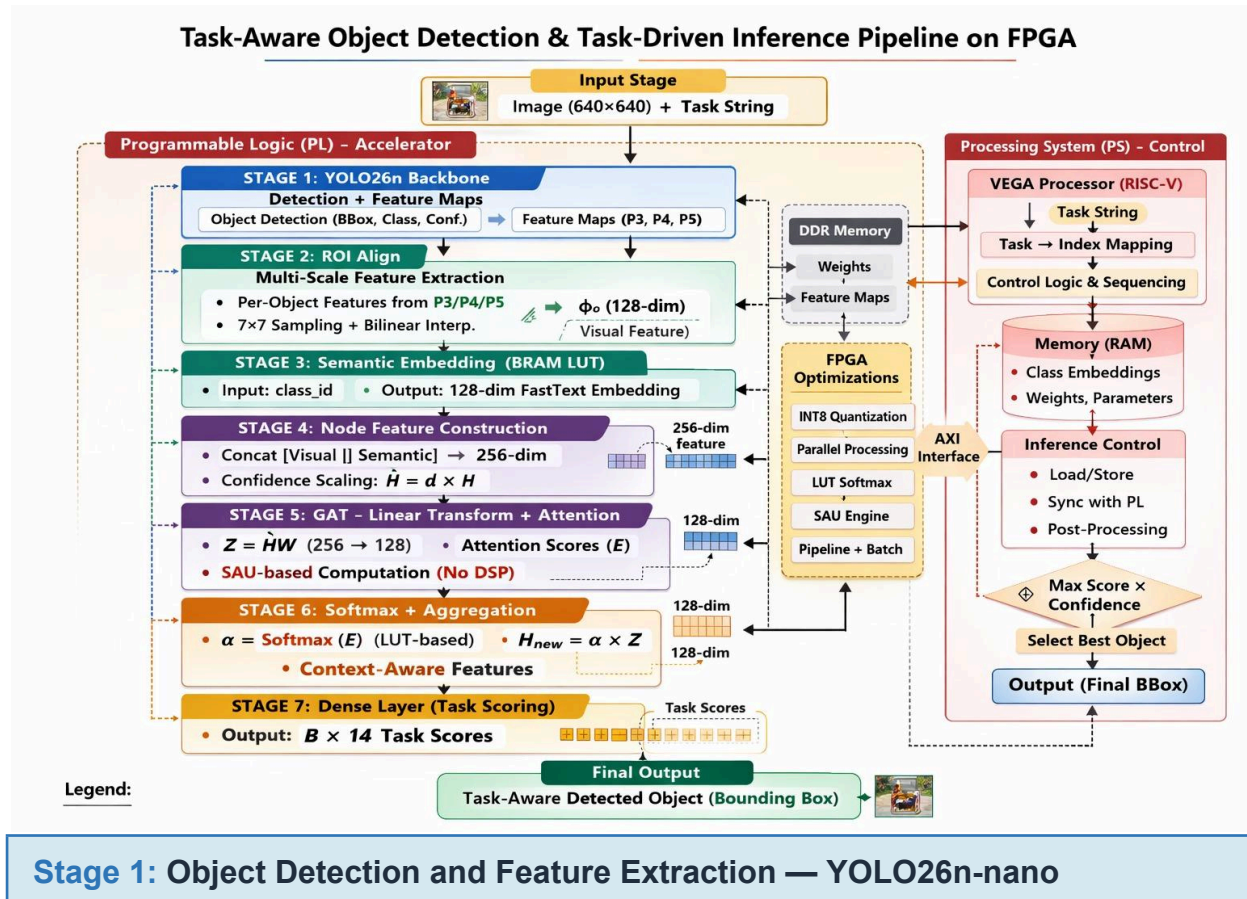
Objectives

1. Design a lightweight single-backbone pipeline in which YOLO26n-nano performs both object detection and feature extraction, with ROI Align extracting per-object visual features directly from neck feature maps, eliminating the need for a separate feature extraction network.
2. Fuse visual features (ϕ_o , 128-dim) with pre-computed FastText semantic class embeddings (128-dim) via concatenation to form 256-dimensional node representations h_o that encode both appearance and semantic meaning.

3. Implement a single-layer Graph Attention Network with confidence-weighted aggregation and joint training across all 14 COCO-Tasks, incorporating an auxiliary discriminator loss during training to improve visual feature discriminability.
4. Convert all model components from float32 to fixed-point integers using calibration on COCO-Tasks training data, targeting INT8 for weights and INT16 for activations, and apply SAU weight compression to the GAT to eliminate DSP48 dependence for graph computation.
5. Design a custom FPGA accelerator using Vitis HLS implementing: YOLO26n convolution engine, ROI Align unit, GAT computation engine with operations fusion and LUT-based softmax, and dense layer scoring, all communicating via BRAM under FSM control.
6. Implement a hardwired Finite State Machine controller on the FPGA programmable logic to orchestrate the pipeline with pipelined object processing, coordinating VEGA processor for task string matching and final score selection.
7. Meet the following targets: inference latency under 100ms, power consumption under 10W, mAP@0.5 of approximately 65-70% on COCO-Tasks, and total model memory under 16MB fitting entirely in on-chip BRAM.

IV. Design Process

The proposed system is a single-backbone, eight-stage pipeline executing across the VEGA processor and custom programmable logic on the Kintex-7 FPGA. Each stage below is described with its specific problem statement, functional specifications, and proposed design.



Problem Statement

The system must process a raw 640×640 RGB image and simultaneously produce two outputs: (1) a list of all detected objects with bounding box coordinates and class labels, and (2) multi-scale intermediate feature maps that encode rich visual information at three different spatial resolutions. Standard object detectors are designed solely for detection output and do not expose intermediate feature representations, while separate

feature extraction networks add prohibitive parameter count and latency for edge deployment. The challenge is to perform both functions in a single forward pass within an FPGA-friendly architecture, eliminating Non-Maximum Suppression to ensure deterministic inference latency.

Functional Specifications

- Input: Single RGB image resized to 640×640 pixels
- Output 1: List of B detected objects, each with bounding box [x, y, w, h], detection confidence score d_i , and class identifier $class_id_i$
- Output 2: Feature maps at three neck layers — P5 (stride 32, shape [1, 256, 20, 20]), P4 (stride 16, shape [1, 128, 40, 40]), P3 (stride 8, shape [1, 64, 80, 80])
- FR-1.1: Process 640×640 input through a 22-layer architecture in a single forward pass
- FR-1.2: Intercept and store feature maps at P3, P4, P5 neck layers via hardware hook mechanism
- FR-1.3: Use One-to-One detection head to eliminate Non-Maximum Suppression for deterministic latency
- FR-1.4: Produce fixed 300-candidate output filtered by configurable confidence threshold on VEGA
- FR-1.5: Fit within approximately 3.2MB of BRAM after INT8 quantization
- FR-1.6: Implement convolutional layers as systolic array of DSP48E1 blocks with pipelined data flow

Proposed Design

YOLO26n-nano serves as the single backbone for both object detection and visual feature extraction, processing the 640×640 input through its 22-layer architecture in a single forward pass. A hardware hook mechanism intercepts and stores output feature maps at three neck layers: P5 (layer ~9, stride 32) encoding semantic features, P4 (layer ~18, stride 16) encoding shape features, and P3 (layer ~15, stride 8) encoding texture and edge features. YOLO26n's One-to-One detection head eliminates Non-Maximum Suppression, producing deterministic latency directly from the FPGA pipeline. After INT8 quantization, YOLO26n-nano occupies approximately 3.2MB of BRAM, and convolutional layers are implemented as a systolic array of DSP48E1 blocks with pipelined data flow.

Stage 2: Per-Object Visual Feature Extraction — ROI Align

Problem Statement

Each detected object occupies a different region of the feature maps produced in Stage 1. To reason about individual objects via a graph network, the system must extract a fixed-size, object-specific visual representation from each of the three feature maps for each of the B detected objects. Bounding box coordinates in pixel space must be mapped to feature map coordinates at each scale, and feature values must be sampled with sub-pixel accuracy (bilinear interpolation) to avoid quantization artifacts. The challenge is to implement this memory-intensive random-access operation efficiently on FPGA, where conventional random access to large feature maps creates bandwidth bottlenecks.

Functional Specifications

- Input: B bounding boxes with pixel coordinates; feature maps P3 [1,64,80,80], P4 [1,128,40,40], P5 [1,256,20,20]
- Output: Per-object 128-dimensional visual fingerprint ϕ_o for each of the B detected objects
- FR-2.1: Divide bounding box pixel coordinates by stride (32, 16, or 8) to obtain feature map coordinates (may be non-integer)
- FR-2.2: Place a 7×7 sampling grid within each feature map region and compute bilinear interpolation at all 49 sample points
- FR-2.3: Concatenate three crops per object along the channel dimension: $[256,7,7] + [128,7,7] + [64,7,7] = [448,7,7]$
- FR-2.4: Apply learned 1×1 convolution to compress $[448,7,7]$ to $[128,7,7]$, then average pool to 128-dim vector ϕ_o
- FR-2.5: Fuse the 1×1 convolution and average pool into a single hardware pass
- FR-2.6: Use line buffers to convert random feature map access into sequential BRAM reads, maximizing memory bandwidth

Proposed Design

For each of the B detected objects, ROI Align extracts fixed-size feature crops from each of the three feature maps by mapping bounding box pixel coordinates to feature map coordinates via stride division. A 7×7 sampling grid is placed within each region and bilinear interpolation computes feature values at each of the 49 sample points from the four surrounding feature map cells. The three crops are concatenated to $[448, 7, 7]$, compressed to $[128, 7, 7]$ via a learned 1×1 convolution, and collapsed via average pooling to produce the 128-dimensional visual fingerprint ϕ_o per object. The 1×1 convolution and average pool are fused into a single hardware pass to minimize memory transactions, and line buffers convert random feature map access into sequential BRAM reads for maximum bandwidth efficiency.

Stage 3: Semantic Feature Retrieval — BRAM LUT

Problem Statement

Visual features alone encode appearance and spatial properties but carry no semantic prior about object class identity and function. The system must augment each detected object's visual representation with a semantic encoding of what that class of object means — its typical usage, functional role, and relationships to other objects. Computing these semantic embeddings at runtime from a language model is infeasible on edge hardware. The challenge is to encode semantic class knowledge at design time, store it efficiently on-chip, and retrieve it in parallel with visual feature extraction to avoid adding pipeline latency.

Functional Specifications

- Input: `class_id_i` for each detected object i (integer, 0–79 for COCO classes)
- Output: 128-dimensional FastText semantic embedding vector for each detected object
- FR-3.1: Pre-compute 128-dimensional FastText embeddings for all 80 COCO class names offline before deployment
- FR-3.2: Quantize embeddings to INT16 fixed-point and store as an 80×128 lookup table in dual-port BRAM (~20KB)
- FR-3.3: Output corresponding 128-dimensional semantic vector in a single clock cycle given `class_id`
- FR-3.4: Pipeline semantic lookup for object i in parallel with ROI Align processing for object i
- FR-3.5: Semantic vectors must encode functional proximity — semantically similar classes (e.g., wine glass and cup) should have smaller vector distances

Proposed Design

Prior to deployment, all 80 COCO class names are encoded offline using FastText into 128-dimensional dense semantic vectors. These vectors are quantized to INT16 fixed-point and stored as an 80×128 lookup table in dual-port BRAM, occupying approximately 20KB. At runtime, the hardware FSM controller sends each detected object's `class_id` to the BRAM LUT module, which outputs the corresponding 128-dimensional semantic vector in a single clock cycle. This operation is fully pipelined with ROI Align: while ROI Align processes the visual features for object i , the semantic lookup for object i occurs simultaneously, adding zero latency to the pipeline. The FastText representation encodes semantic proximity between classes, providing richer prior knowledge than the one-hot class encoding used in the original COCO-Tasks reference.

Stage 4: Node Feature Construction and Confidence Scaling

Problem Statement

The visual fingerprint ϕ_o from Stage 2 and the semantic embedding from Stage 3 are complementary representations — one captures what the object looks like in this specific image, the other captures what objects of this class typically do. These must be combined into a single unified node feature vector for the GAT. Additionally, not all detected objects are equally reliable: low-confidence detections from Stage 1 introduce noise that can corrupt graph reasoning. The challenge is to merge the two feature types efficiently and propagate detection confidence through the entire downstream computation so that uncertain detections exert proportionally less influence on all subsequent processing.

Functional Specifications

- Input: Visual fingerprint ϕ_o (128-dim) and semantic embedding (128-dim) for each object; detection confidence score d_i
- Output: Confidence-scaled node representation $\tilde{H} \in R^{(B \times 256)}$ for each of the B detected objects
- FR-4.1: Concatenate ϕ_o (128-dim) with semantic embedding (128-dim) via adjacent BRAM addressing — no arithmetic computation required
- FR-4.2: Produce intermediate 256-dimensional node representation $H = [\phi_o \parallel \text{semantic}] \in R^{(B \times 256)}$
- FR-4.3: Apply confidence scaling: $\tilde{H} = d \cdot H$ where $d \in R^{(B \times 1)}$ (row-wise multiplication by scalar d_i)
- FR-4.4: Implement confidence scaling as parallel multiply unit using SAU shift operations (no DSP48 usage)
- FR-4.5: Low-confidence objects must produce weaker node representations that exert proportionally less influence on attention scores and aggregated representations

Proposed Design

Visual and semantic features are concatenated via adjacent BRAM addressing — placing the two 128-number blocks sequentially without arithmetic computation — to form the 256-dimensional node representation H . YOLO26n confidence scores are then applied as row-wise scaling: $\tilde{H} = d \cdot H$, scaling each object's entire 256-dimensional feature vector by its detection confidence d_i . This propagates the confidence signal through the entire GAT computation, naturally down-weighting all contributions from uncertain detections. Low-confidence objects produce weaker node representations that exert less influence on attention scores and aggregated representations downstream. The confidence-weighted scheme, validated by Sawatzky et al. (2019) to improve mAP from 29.3% to 30.3% on detections, is implemented as a parallel multiply unit using SAU shift operations — requiring no DSP48 blocks.

Stage 5: GAT Linear Transform and Attention Score Computation

Problem Statement

The 256-dimensional node representations from Stage 4 must be projected into a lower-dimensional space optimized for graph reasoning, and pairwise attention scores must be computed between all pairs of objects to quantify how much each object should attend to each other. Standard GAT implementations perform the linear transform and attention computation in separate sequential steps, each requiring a full BRAM read/write cycle, creating memory synchronization overhead. Additionally, the weight matrices for the linear transform contain float32 multiplications that would consume all available DSP48 blocks if implemented conventionally, leaving none for the YOLO26n convolution engine.

Functional Specifications

- Input: Confidence-scaled node representations $\tilde{H} \in R^{(B \times 256)}$
- Output: Raw attention matrix $E \in R^{(B \times B)}$; transformed node features $Z \in R^{(B \times 128)}$
- FR-5.1: Apply linear transform: $Z = \tilde{H} \cdot W$ where $W \in R^{(256 \times 128)}$, compressing each node from 256 to 128 dimensions
- FR-5.2: Split attention vector $a \in R^{256}$ into a_{src} and a_{dst} (each 128-dim), compute $e_{\text{src}} = Z \cdot a_{\text{src}}$ and $e_{\text{dst}} = Z \cdot a_{\text{dst}}$ during the linear transform pass (operations fusion)
- FR-5.3: Compute raw attention matrix via broadcast: $E = \text{ReLU}(e_{\text{src}} \cdot 1^T + 1 \cdot e_{\text{dst}}^T) \in R^{(B \times B)}$
- FR-5.4: Use ReLU (not LeakyReLU) activation — fine-tuning with ReLU must recover accuracy to within 0.5% of LeakyReLU baseline
- FR-5.5: Implement all multiplications in W , a_{src} , and a_{dst} using Shift Addition Units (SAU) — weights compressed to powers of two — achieving zero DSP48 usage for the entire GAT computation
- FR-5.6: Operations fusion must reduce memory traffic by approximately 40% compared to the standard three-step GAT

Proposed Design

The GAT linear transform compresses each node from 256 to 128 dimensions: $Z = \tilde{H} \cdot W$ where $W \in R^{(256 \times 128)}$. Following the operations fusion technique from FPGAN (Yan et al., 2020), attention score

computation is fused into the linear transform module. The attention vector $a \in \mathbb{R}^{256}$ is split into a_{src} and a_{dst} , applied to the transformed source and destination nodes during the same pass, producing e_{src} and e_{dst} vectors without a separate BRAM read cycle. The raw attention matrix is then computed: $E = \text{ReLU}(e_{src} \cdot 1^T + 1 \cdot e_{dst}^T)$. ReLU replaces LeakyReLU following FPGAN's verified finding that fine-tuning recovers accuracy to within 0.5% while eliminating the $0.2\times$ multiplication for negative values. All multiplications use Shift Addition Units — weights compressed to powers of two via iterative calibration — eliminating DSP48 dependence for the entire GAT and freeing all 840 DSP48 blocks for the YOLO26n convolution accelerator.

Stage 6: Softmax Normalization and Weighted Aggregation

Problem Statement

The raw attention scores from Stage 5 must be normalized row-wise via softmax to produce valid probability distributions over neighboring nodes, and then used to compute weighted sums of transformed features to produce context-enriched node representations. The critical FPGA challenge is the softmax operation: computing $\exp()$ for floating-point values requires either lookup tables that are too large, or dedicated arithmetic units that consume DSP48 resources already reserved for YOLO26n. The aggregation step itself involves a matrix multiplication over B objects that must be pipelined efficiently to meet latency targets.

Functional Specifications

- Input: Raw attention matrix $E \in \mathbb{R}^{(B \times B)}$; transformed node features $Z \in \mathbb{R}^{(B \times 128)}$
- Output: Context-enriched node representations $H_{new} \in \mathbb{R}^{(B \times 128)}$ encoding inter-object relational awareness
- FR-6.1: Apply row-wise softmax normalization: $\alpha = \text{softmax_row}(E) \in \mathbb{R}^{(B \times B)}$
- FR-6.2: Implement $\exp()$ via LUT-based approximation — pre-compute $\exp()$ values for the valid range of attention scores during calibration; store in approximately 160 bytes of BRAM
- FR-6.3: At runtime, quantize each attention score to a fixed-point index and retrieve the corresponding $\exp()$ value in 1–2 clock cycles; zero DSP48 usage
- FR-6.4: Compute feature aggregation: $H_{new} = \alpha \cdot Z \in \mathbb{R}^{(B \times 128)}$
- FR-6.5: Each row of H_{new} must encode awareness of all other objects in the scene — the mechanism enabling context-dependent task-driven selection

Proposed Design

Row-wise softmax normalizes the attention matrix: $\alpha = \text{softmax_row}(E) \in \mathbb{R}^{(B \times B)}$. Following FPGAN's LUT-based approximation, $\exp()$ values for the valid range of attention scores are pre-computed during calibration and stored in approximately 160 bytes of BRAM. At runtime, each attention score is quantized to a fixed-point index and the corresponding $\exp()$ value is retrieved in one to two clock cycles without any DSP usage. Feature aggregation then produces context-enriched node representations: $H_{new} = \alpha \cdot Z \in \mathbb{R}^{(B \times 128)}$, where each row of H_{new} is a weighted combination of all objects' transformed features. The weights reflect both learned task relevance (from attention scores) and detection confidence (propagated from Stage 4), enabling each object's representation to encode awareness of what other objects are present — the core mechanism for context-dependent task-driven selection.

Stage 7: Dense Layer — 14 Task Suitability Scores

Problem Statement

The context-enriched node representations from Stage 6 encode relational awareness across all detected objects but must be projected into a task-specific scoring space. The system must score each object's suitability for each of the 14 COCO-Tasks, producing a $B \times 14$ logit matrix. The challenge is that training separate task-specific models would require $14\times$ the parameters and training data, while a single jointly-trained model must learn representations that are simultaneously discriminative for all 14 tasks. The weight matrix for this dense layer must also avoid DSP48 usage to maintain the hardware resource budget.

Functional Specifications

- Input: Context-enriched node representations $H_{\text{new}} \in \mathbb{R}^{(B \times 128)}$
- Output: Task logit matrix $\text{Logits} \in \mathbb{R}^{(B \times 14)}$, one row per detected object and one column per task
- FR-7.1: Apply linear layer: $\text{Logits} = H_{\text{new}} \cdot W_{\text{out}}^T + b_{\text{out}}$ where $W_{\text{out}} \in \mathbb{R}^{(14 \times 128)}$, $b_{\text{out}} \in \mathbb{R}^{14}$
- FR-7.2: W_{out} and b_{out} must use SAU-compressed weights — zero DSP48 usage
- FR-7.3: Train jointly across all 14 COCO-Tasks simultaneously with shared GAT weights — joint training must improve performance by at least 3.1% mAP compared to task-specific training (per Sawatzky et al., 2019)
- FR-7.4: Apply auxiliary discriminator loss during training (128→64→14 MLP on ϕ_o features) to improve visual feature discriminability from 74.2% to 77.1% mAP; discard this auxiliary network entirely at inference

Proposed Design

A learned linear layer maps each object's context-enriched representation to 14 task suitability scores: $\text{Logits} = H_{\text{new}} \cdot W_{\text{out}}^T + b_{\text{out}}$ where $W_{\text{out}} \in \mathbb{R}^{(14 \times 128)}$ and $b_{\text{out}} \in \mathbb{R}^{14}$. The output $\text{Logits} \in \mathbb{R}^{(B \times 14)}$ contains one row per detected object and one column per task. The model is trained jointly across all 14 tasks with shared GAT weights, which Sawatzky et al. (2019) showed improves performance by 3.1% mAP compared to task-specific training. During training, an auxiliary discriminator MLP (128→64→14) applied directly to ϕ_o features forces visual representations to be task-discriminative, improving ground truth accuracy from 74.2% to 77.1% mAP; this auxiliary network is completely discarded at inference. W_{out} and b_{out} use SAU-compressed weights, maintaining zero DSP48 usage throughout the entire GAT module.

Stage 8: Task Selection — VEGA Processor

Problem Statement

The $B \times 14$ logit matrix from Stage 7 contains task suitability scores for every detected object across all 14 tasks. The system must select the single most suitable object for the specific input task string. This requires mapping the free-text input task description to the correct task column, multiplying scores by detection confidence for final ranking, and returning the bounding box of the winning object. This control flow, string matching, and final selection logic is software-friendly but hardware-unfriendly, making the VEGA RISC-V processor the natural execution environment for this stage.

Functional Specifications

- Input: Task logit matrix $\text{Logits} \in \mathbb{R}^{(B \times 14)}$ received via AXI bus; input task description string; YOLO detection confidence scores
- Output: Bounding box coordinates [x, y, width, height] and class label of the most suitable detected object
- FR-8.1: Map input task string to `task_index` (0–13) via exact string comparison against 14 predefined COCO-Tasks descriptions — must complete in microseconds on VEGA
- FR-8.2: Extract column `task_index` from Logits matrix
- FR-8.3: Multiply each object's score by its corresponding YOLO detection confidence
- FR-8.4: Find the index of the maximum confidence-weighted score
- FR-8.5: Return bounding box and class label of the winning object as final pipeline output
- FR-8.6: No neural network processing of task text during contest evaluation — all 14 task embeddings are known in advance and `task_index` is resolved via trivial C string comparison

Proposed Design

The VEGA processor receives the Logits matrix via AXI bus and executes a simple selection loop: (1) perform exact string matching of the input task description against the 14 predefined COCO-Tasks descriptions to determine `task_index`, requiring only microseconds as a trivial C string comparison; (2) extract column `task_index` from the Logits matrix; (3) multiply each score by the corresponding YOLO detection confidence for final confidence-weighted ranking; (4) find the index of the maximum score; and (5) return the bounding box and class label of the winning object. No neural network processes the task string during inference, as all 14

task descriptions are fixed at design time. VEGA's RISC-V instruction set handles this sequential control flow naturally while the custom PL accelerators handle all compute-intensive operations.

V. Training Methodology

The complete pipeline is trained end-to-end on the COCO-Tasks dataset (30,229 training images, 9,495 test images) in three phases:

- Phase 1: Train with LeakyReLU activations, auxiliary discriminator loss on ϕ_o features, and joint binary cross-entropy loss across all 14 tasks. The auxiliary discriminator forces visual features to be task-discriminative, improving ground truth accuracy from 74.2% to 77.1% mAP.
- Phase 2: Replace LeakyReLU with ReLU and fine-tune to recover accuracy to within 0.5% of Phase 1 levels.
- Phase 3: Compress GAT weights to powers of two using FPGAN's iterative compression algorithm; fine-tune to maintain accuracy within 0.5% of Phase 2 levels.

VI. Resource Budget

All model weights fit within the 16MB BRAM with a comfortable margin, and input images are the only data streamed from DDR3. The estimated resource utilization is summarized below:

Component	DSP48	BRAM	LUTs (est.)	Utilization
YOLO26n conv accelerator	~528	~9MB	~32,000	~39%
ROI Align unit (3 scales)	~50	~5MB	~15,000	
1×1 conv compression	~30	~0	~5,000	~58%
GAT accelerator (SAU-based)	~0	~46KB	~30,000	
VEGA soft processor	~100	~1MB	~15,000	~54%
AXI interconnect	~0	~0	~5,000	
Total estimated	~708	~9.3MB	~102,000	
Available (Kintex-7)	840	16MB	203,150	

YOLO26n (~9MB), 1×1 conv weights (~57KB), GAT weights W and a (~33KB), dense layer W_{out} (~7KB), FastText semantic LUT (~20KB), and $\exp()$ LUT (~160B) all fit within the 16MB BRAM.

VII. PS/PL Division of Work

The VEGA processor (PS) handles: receiving input image and task string; task string matching; filtering YOLO detections by confidence threshold; writing confidence scores to PL; signaling each PL accelerator module to start; reading final logits; selecting the winner; outputting the bounding box.

The custom programmable logic (PL) handles: all YOLO26n convolution layers; feature map interception at P3, P4, P5; ROI Align with bilinear interpolation; 1×1 conv and average pool; semantic BRAM LUT retrieval; confidence scaling; GAT linear transform with operations fusion; attention matrix computation; LUT-based softmax; weighted aggregation; dense layer scoring.

The pipeline controller is implemented as a hardwired Finite State Machine on PL, coordinating pipelined processing of multiple objects: while ROI Align processes object i , the semantic lookup for object $i+1$ occurs simultaneously, reducing effective per-object processing time to approximately one stage per clock cycle after the initial pipeline fill.

References

- Sawatzky, J. et al. (2019). What Object Should I Use? Task Driven Object Detection. arXiv:1904.03000
- Yan, W. et al. (2020). FPGAN: An FPGA Accelerator for Graph Attention Networks With Software and Hardware Co-Optimization. IEEE Access.
- Velickovic, P. et al. (2018). Graph Attention Networks. ICLR 2018. arXiv:1710.10903
- Amin, R. A. et al. (2024). FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing. IEEE Access, vol. 12.
- Amin, R. A. et al. (2024). FPGA-based Resource Efficient High Throughput Object Detection Using Pipelined CNN and Custom SSD. IEEE NorCAS 2024.
- Ren, S. et al. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS 2015.
- He, K. et al. (2017). Mask R-CNN. ICCV 2017.
- Bojanowski, P. et al. (2017). Enriching Word Vectors with Subword Information (FastText). TACL.
- YOLO26 (2026). Ultralytics YOLO26 Technical Report.
- Dremov, A. et al. (2026). Compute-Optimal Quantization-Aware Training. arXiv:2509.22935